



Politecnico
di Torino

Generative Artificial Intelligence for graphics and multimedia

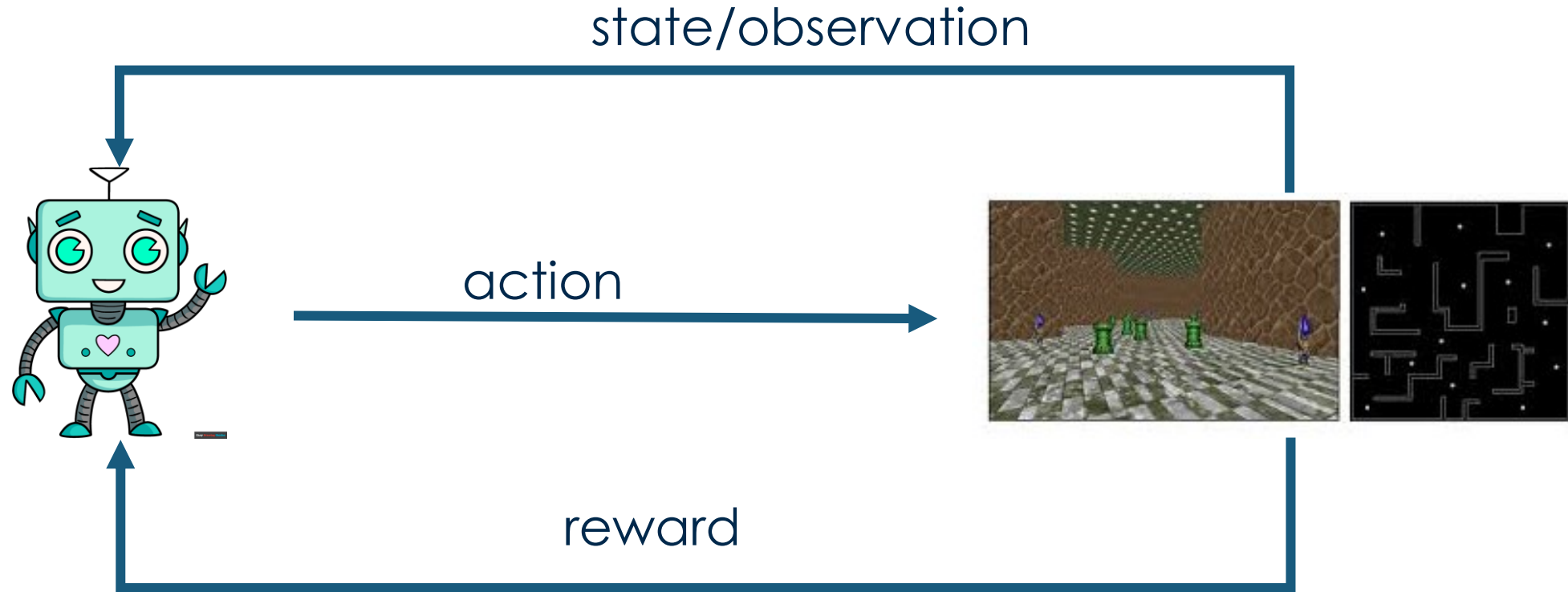
Reinforcement learning
Prof. Lia Morra



What is reinforcement learning?

- People and animals learn by **interacting with our environment**
 - This differs from certain other types of learning
 - It is **active** rather than passive
- Interactions are often **sequential** — future interactions can depend on earlier ones
- We are **goal-directed**
- We can learn **without examples** of optimal behaviour
- Instead, we optimise some **reward signal**

The interaction loop



Goal: optimise sum of rewards, through repeated interaction

However: we cannot learn via gradient descent because the environment is not differentiable

Reward hypothesis

Reward hypothesis: “ that the generic objective of maximising reward is enough to drive behaviour that exhibits most if not all abilities that are studied in natural and artificial intelligence.”

Silver, David, et al. "Reward is enough." *Artificial intelligence* 299 (2021): 103535.

Any goal can be formalized as the outcome of maximizing a cumulative reward

Why reinforcement learning?

Find **solutions**

- a program that plays chess really well
- a Non-Playing Character (NPC) that behaves intelligently

2. **Adapt online**, deal with unforeseen circumstances

- a chess program that can learn to adapt to you
- a robot that can learn to navigate unknown terrains
- a chatbot that tailors the style of its response to individual users

Reinforcement learning can provide **algorithms** for both cases

- Note that the second point is not (just) about generalization — it is about continuing to learn efficiently online, during operation

Robotics



PRODUCTION



DeepMind

STAR CRAFT
DEMONSTRATION

40
← REPLAY

Catalyst LE



3 AlphaStar



112 / 126

305 +1651

556 +671

48

64

520

SUPPLY

MINERALS

GAS

WORKERS

ARMY

APM

2 LiquidMeNo



00 / 110

650 +1520

240 +640

46

46

295

Workers: 16/16

BILZARD
ENTERTAINMENT



<https://www.youtube.com/watch?v=IPERfjRaZug>

RLHF — RL from Human Feedback

The critical post-training step that turns a language model into an assistant: ChatGPT, Claude, Gemini all use RLHF (or variants like DPO) to align outputs with human preferences.

RL vs Supervised Learning

SL learns from fixed labeled data.
RL learns from interaction with an environment — no labels, only reward signals.

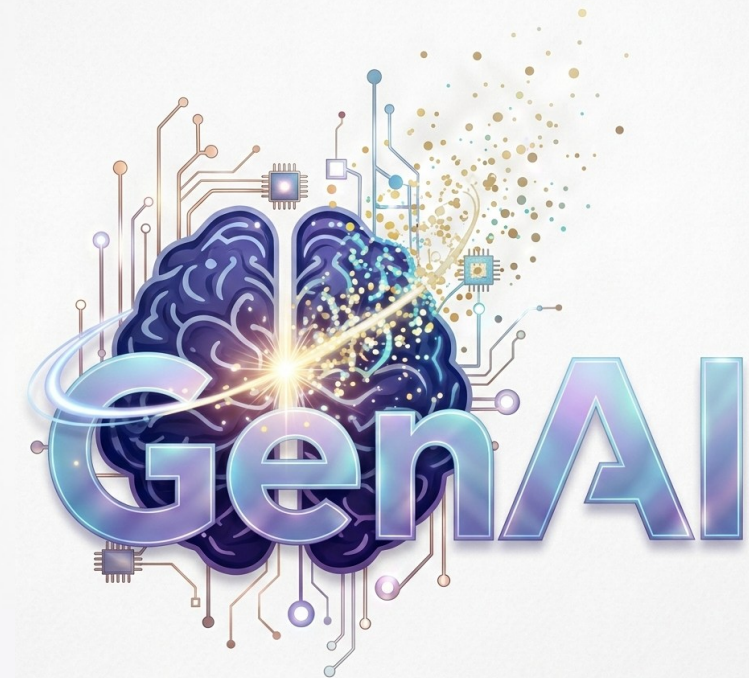
RL vs Unsupervised Learning

UL finds structure in data.
RL optimizes a goal over time: the agent's decisions affect future observations.



Politecnico
di Torino

Framing the problem



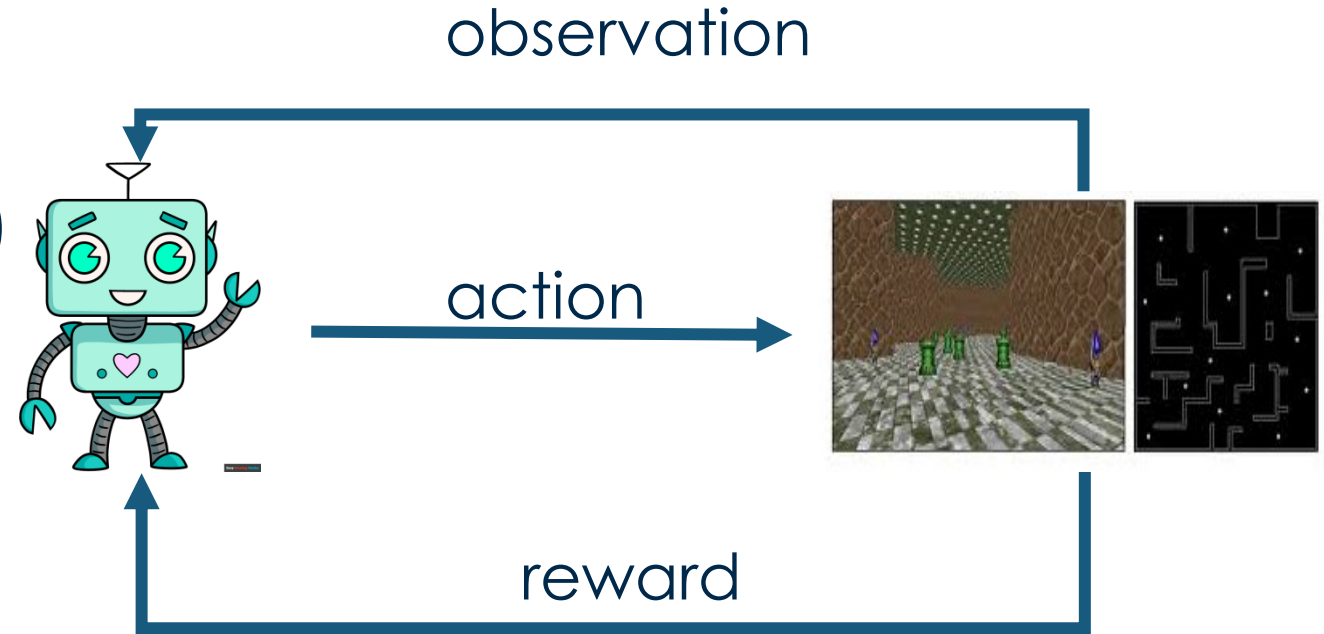
Agent and environment

At each step t the agent:

- receives observation O_t (and reward R_t)
- executes action A_t

The environment:

- receives action A_t
- emits observation O_{t+1} (and reward R_{t+1})



A **reward** R_t is a scalar feedback signal: indicates how well agent is doing at step t — defines the goal

The agent's job is to maximize cumulative reward or **return**

$$G_t = R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

Value

- We call the expected cumulative reward, from a state s and action a , the **value**

$$v(s) = \mathbb{E}[G_t \mid S_t = s] = \mathbb{E}[R_{t+1} + R_{t+2} + R_{t+3} + \dots \mid S_t = s]$$

- The value depends on the actions the agent takes
- Goal is to **maximize value**, by picking suitable actions
- Rewards and values define **utility** of states and action (no supervised feedback)
- Returns and values can be defined recursively

$$G_t = G_{t+1} + R_{t+1}$$

$$V(s) = \mathbb{E}[v(S_{t+1}) + R_{t+1} \mid S_t = s]$$

Exploration vs. exploitation

Goal: select actions to maximise value

- Actions may have long term consequences
- Reward may be delayed
- When planning: decisions involve reasoning about not just immediate benefit of a decision but also its longer term ramifications
- When learning: temporal credit assignment is hard (what caused later high or low rewards?)

Exploitation: maximise performance based on current knowledge

Exploration: learning about the world by making decisions

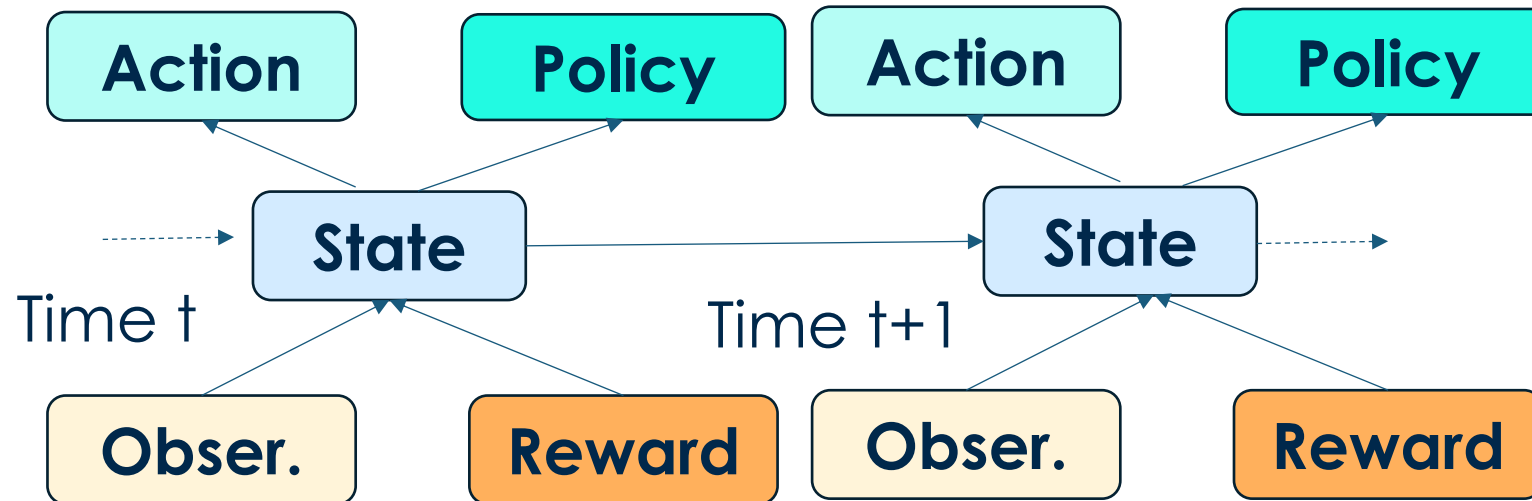
- The best long-term strategy may involve short-term sacrifices

Agent components

The **history** is the full sequence of observations, actions, rewards

$$H_t = O_0; A_0; R_1; O_1; \dots; O_{t+1}; A_{t+1}; R_t; O_t$$

This history is used to construct the **agent state** S_t



Environment state

The **environment state** is the environment's internal state

- It is usually invisible to the agent
- Even if it is visible, it may contain lots of irrelevant information

Fully observable environment

An NPC has perfect knowledge of the game state

The agent sees the full environment state
→ observation = environment state
→ $S_t = O_t$ = environment state

Partially observable environment

A robot with camera vision does not know its absolute location

The agent has limited visibility
→ observation $\neq$ environment state

Markov Decision Process (MDP)

An MDP is a tuple (S, A, P, R, γ) — the mathematical description of an RL problem

S

State Space

All possible situations the agent can be in.
Example: board configuration, robot joint angles

A

Action Space

All decisions the agent can take.
Example: {left, right, up, down}, continuous torque values.

P

Transition Model

$P(s' | s, a)$ — probability of landing in s' after taking action a in state s .

R

Reward Function

$R(s, a)$ — scalar feedback signal.
Encodes the goal: what the agent should maximise.

γ

Discount Factor

$\gamma \in [0,1]$ — how much the agent cares about future rewards vs immediate ones.

RL and Markov properties

The Markov Property

"The future is independent of the past, given the present state."

Formally: $P(s_{t+1} | s_0, \dots, s_t) = P(s_{t+1} | s_t)$

Why it matters:

- Tractable computation — we only need current state, not full history
- Most RL algorithms are designed for MDPs)

POMDP: agent receives partial observation o_t . Must maintain belief state $b(s) = P(s | o_1, \dots, o_t)$. Much harder!

Policy $\pi(a | s)$

The agent's brain — maps states to actions.

Deterministic

$$\pi(s) \rightarrow a$$

Outputs a single action per state. Simple, efficient. Example: always go right.

Stochastic

$$\pi(a | s) \rightarrow [0,1]$$

Outputs a probability distribution. Supports exploration. Essential for continuous control.

Value function

State Value Function $V^\pi(s)$

Expected cumulative reward starting from state s , following policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s]$$

Action Value Function $Q^\pi(s, a)$

Expected cumulative reward taking action a in state s , then following π :

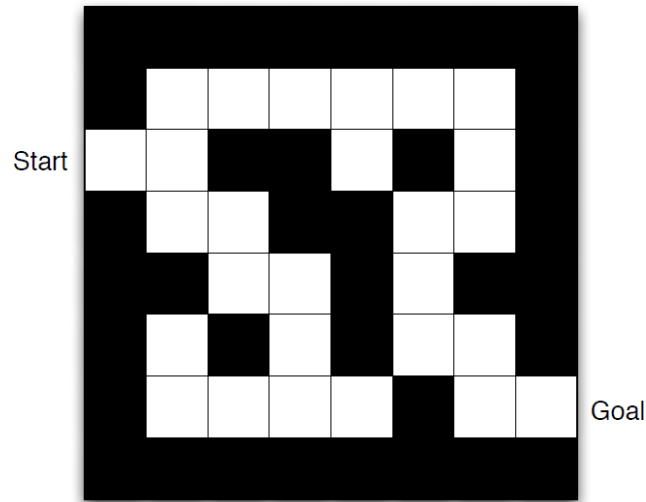
$$\begin{aligned} Q^\pi(s, a) \\ = \mathbb{E}_\pi [r_t + \gamma Q^\pi(s_{t+1}, a_{t+1}) \mid s_t = s, A_t \sim \pi(s)] \end{aligned}$$

Bellman Optimality Equations

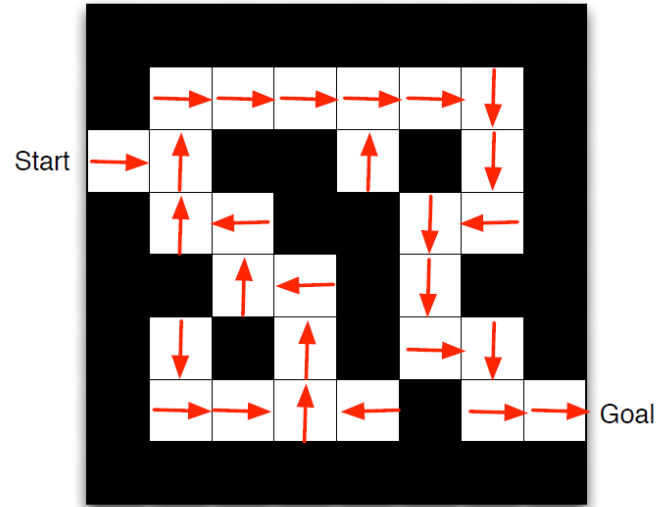
$$\begin{aligned} V^*(s) &= \max_a \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma V^*(s')] \\ Q^*(s, a) &= \sum_{s'} P(s' \mid s, a) [R(s, a) + \gamma \max_{a'} Q^*(s', a')] \end{aligned}$$

If we know Q^* , the optimal policy is trivially: $\pi^*(s) = \arg \max_a Q^*(s, a)$

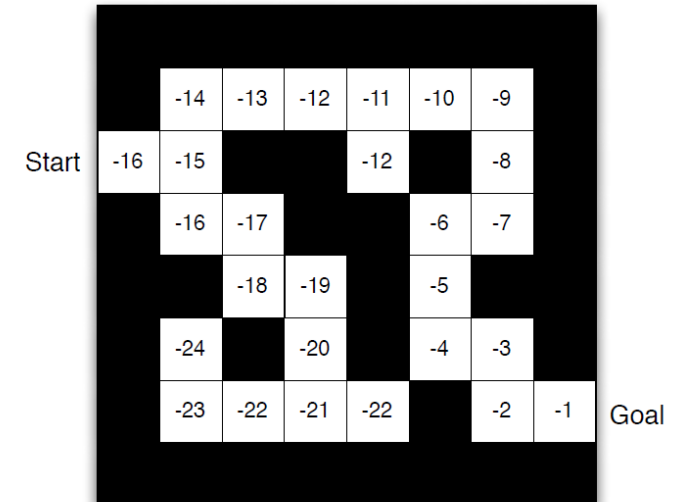
Example



Rewards: -1 per time-step
Actions: N, E, S, W
States: Agent's location



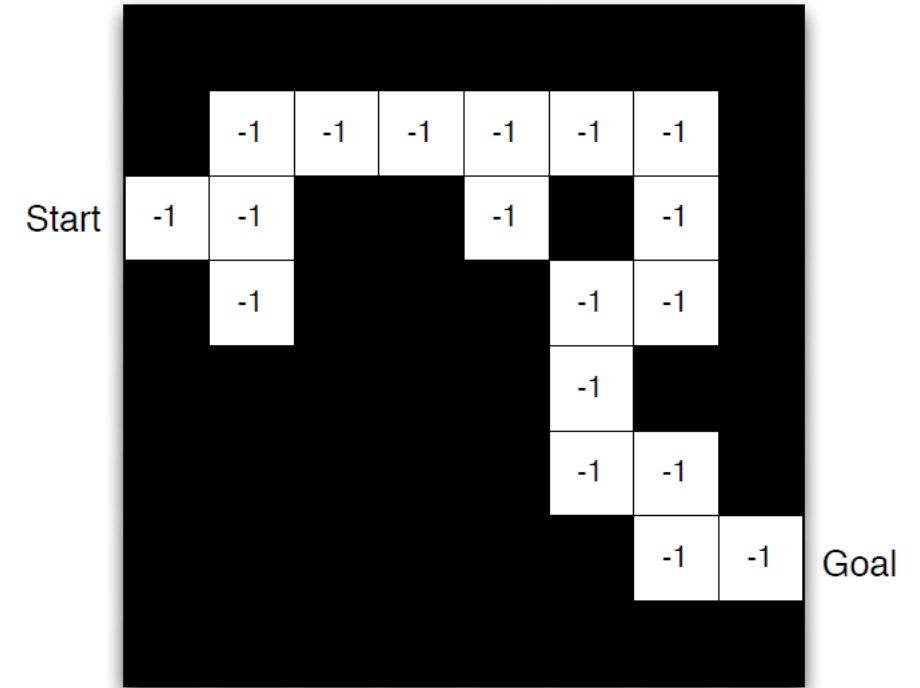
Arrow: policy for each state



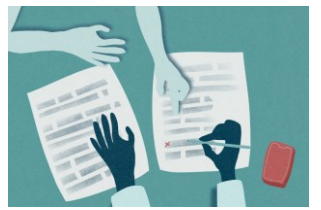
Value for each state

Example

- The grid layout represents partial transition model Pa
- Numbers represent immediate reward Ra from each state s (same for all a and s in this case)



Discuss: a possible agent design for...



 Chess / Board Game

 Robot Arm Control

 LLM Fine-tuning (RLHF)

S

State Space



A

Action Space



P

Transition Model



R

Reward Function



Formalizing the problem: multi-armed bandit

A multi-armed bandit is a **set of distributions** $\{\mathcal{R}_a \mid a \in \mathcal{A}\}$

- $\mathcal{A}$ is a (known) set of **actions** (or “arms”)
- $\mathcal{R}_a$ is a distribution on **rewards**, given action a
- At each step t the agent selects an action $A_t \in \mathcal{A}$
- The environment generates a reward $R_t \sim \mathcal{R}_{A_t}$

The goal is to **maximise cumulative reward** $\sum_{i=1}^t R_i$

- We do this by learning a **policy**: a distribution on $\mathcal{A}$
- We denote the policy as function $\pi_t(a)$

Values and regret

The **action value** for action a is the expected reward

$$q(a) = \mathbb{E}[R_t \mid A_t = a]$$

A simple estimate is the average of the sampled rewards up to time t . It can also be updated incrementally

- The **optimal value** is

$$v^* = \max_{a \in \mathcal{A}} q(a) = \max_{a \in \mathcal{A}} \mathbb{E}[R_t \mid A_t = a]$$

- The **regret** of an action a is

$$\Delta_a = v^* - q(a)$$

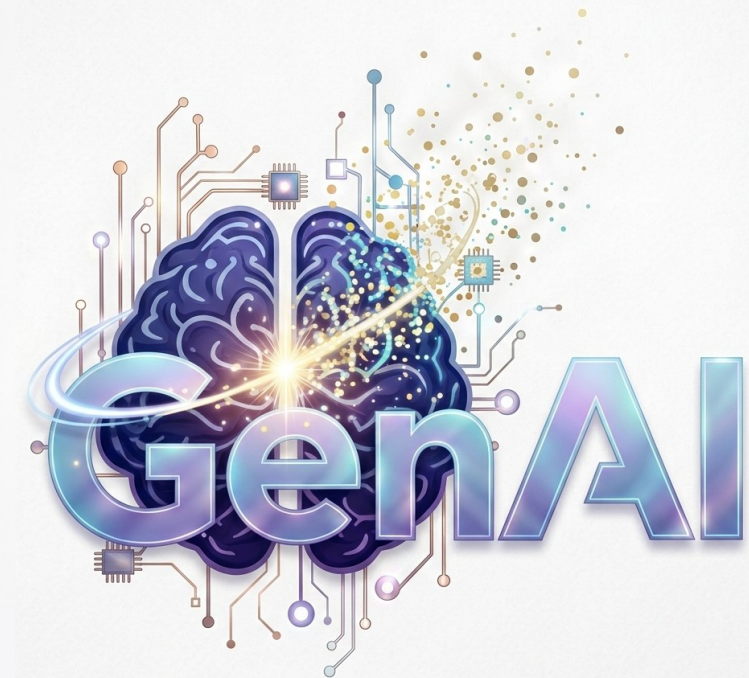
- The regret for the optimal action is zero
- We want to minimise **total regret**:

$$\mathcal{L}_t = \sum_{i=1}^t v^* - q(A_i) = \sum_{i=1}^t \Delta_{A_i}$$



Politecnico
di Torino

Deep reinforcement learning



Learning agent components

All components of an agent are potentially **functions**:

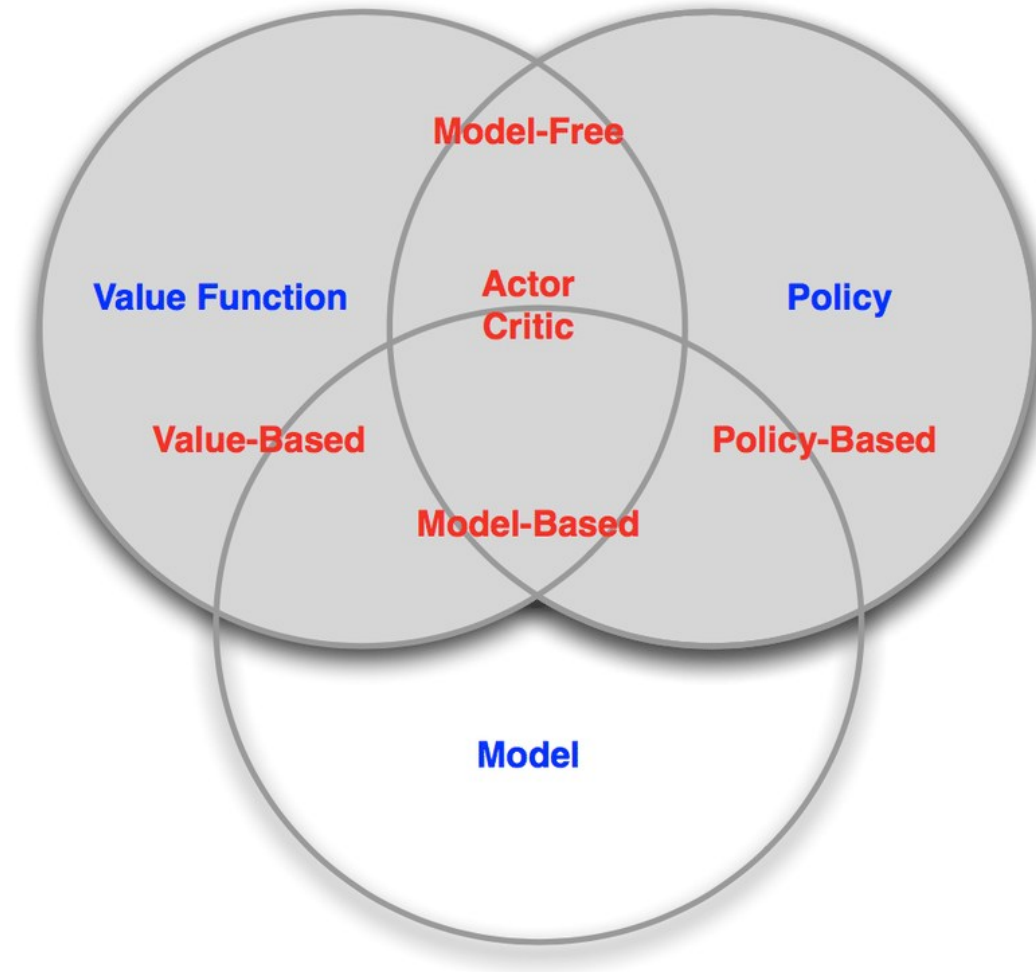
- Policy $\pi: \mathcal{S} \rightarrow \mathcal{A}$ (or probabilities over this space)
- Value functions $v: \mathcal{S} \rightarrow \mathbb{R}$
- State update $u: \mathcal{S} \times \mathcal{O} \rightarrow \mathcal{S}$
- Models $m: \mathcal{S} \rightarrow \mathcal{S}$ or $r: \mathcal{S} \rightarrow \mathbb{R}$

Each of this component can be

- deterministic or stochastic
- hard-coded or learnt
- exact or approximated

If there are too many states, we need to **approximate**, e.g., we can use neural networks to learn them → **deep reinforcement learning**

Agent categories



Learning vs. planning

Two fundamental problems in reinforcement learning

Learning:

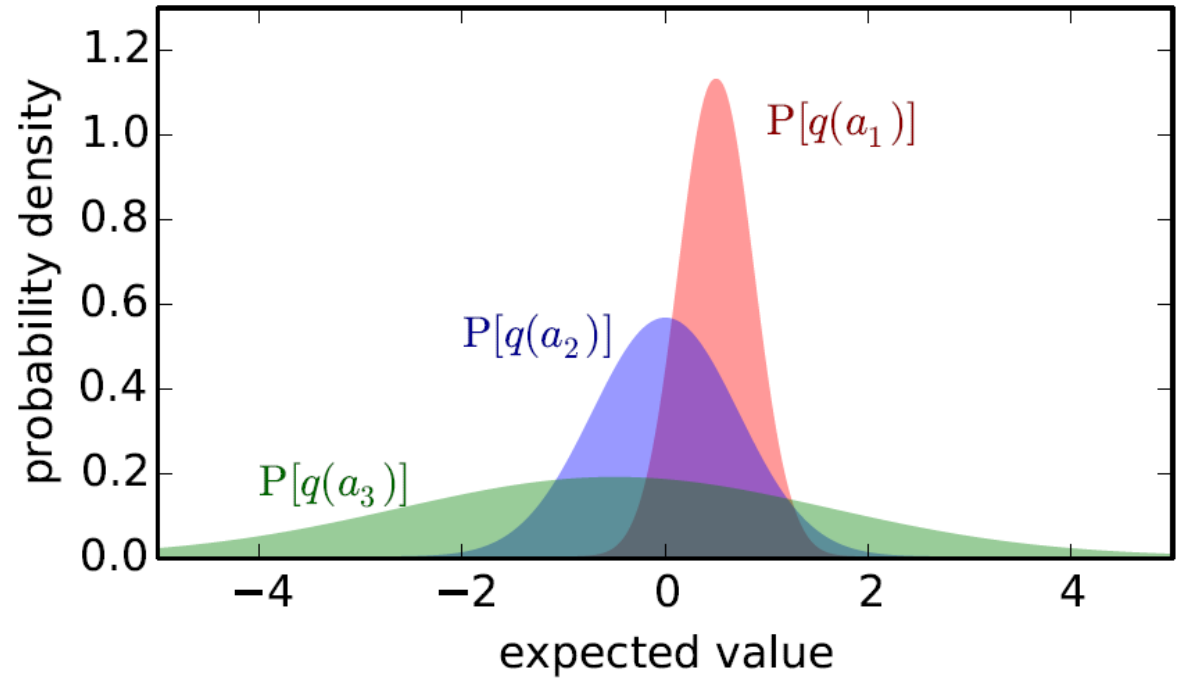
- The environment is initially unknown
- The agent interacts with the environment

Planning:

- A model of the environment is given (or learnt)
- The agent plans in this model (without external interaction)
a.k.a. reasoning, pondering, thought, search, planning

Agent policy

One of the simplest policies is **greedy**: select action with highest value. However, policy is typically a learnt function.



Which action should we pick? More uncertainty about its value: more important to explore that action (optimism in the face of uncertainty)

Exploration vs. exploitation dilemma

ϵ -Greedy

✓ Simple, widely used.
 ϵ can decay: $\epsilon = \epsilon_0 / \sqrt{t}$

$a = \{ \underset{a \in \mathcal{A}}{\operatorname{argmax}} Q(s, a) \text{ with prob. } 1 - \epsilon$
random action with prob. ϵ

✗ Explores uniformly — wastes tries on clearly bad actions

UCB (Upper Confidence Bound)

✓ Optimism in the face of uncertainty.
Explores less-visited actions

$a = \underset{a \in \mathcal{A}}{\operatorname{argmax}} [Q(s, a) + c \sqrt{\log t / N_t(a)}]$

✗ Hard to apply to large/continuous action spaces

Boltzmann / Softmax

✓ Smooth exploration; temperature τ controls randomness

$\pi(a|s) = \exp(Q(s, a)/\tau) / \sum \exp(Q(s, a')/\tau)$

✗ Sensitive to Q-value scale; τ requires tuning

↑ higher exploration → better long-term policy

↓ lower exploration → faster convergence, risk of local optima

Tabular RL: Q-Learning vs. SARSA

Q-Learning (Off-Policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

- Off-policy: learns Q^* regardless of behaviour policy
- Tabular: Q stored as $|S| \times |A|$ table
- ϵ -greedy behaviour: explore randomly with prob ϵ
- Converges to Q^* under mild conditions (all (s,a) visited infinitely)

Key insight: always bootstraps from best next action, even if agent doesn't take it

SARSA (On-Policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

- On-policy: learns Q for the policy actually followed
- Name: State–Action–Reward–State'–Action' (s,a,r,s',a')
- More conservative: penalised for risky actions taken
- Better when exploration policy must be safe

Key insight: learns about the policy it executes, including exploration mistakes

Deep RL: approximating action preferences

Problem: Tabular Q-learning fails for large/continuous state spaces (e.g., Atari pixels). Solution: approximate $Q(s,a;\theta)$ with a neural network.

Experience Replay Buffer

Store transitions (s,a,r,s') in buffer D
Sample random minibatches to train

- Breaks temporal correlation between samples
- Each transition reused multiple times
- More stable, data-efficient training

Target Network

Separate network θ^- for computing targets

Updated every N steps: $\theta^- \leftarrow \theta$

- Stabilizes training (fixed target for updates)
- Without it: chasing a moving target → divergence

Loss (Huber / Smooth L1): $L(\theta) = \mathbb{E}[\delta(r + \gamma Q(s', a'; \theta^-) - Q(s, a; \theta))]$

$\delta(x) = \{ \frac{1}{2}x^2 \text{ if } |x| \leq 1 \mid |x| - \frac{1}{2} \text{ otherwise} \}$ — robust to large temporal difference errors

Policy gradient optimization

Motivation: Q-learning requires $\operatorname{argmax}_{a \in \mathcal{A}}$ → infeasible for continuous or high-dimensional action spaces → directly optimize parameters θ of the policy.

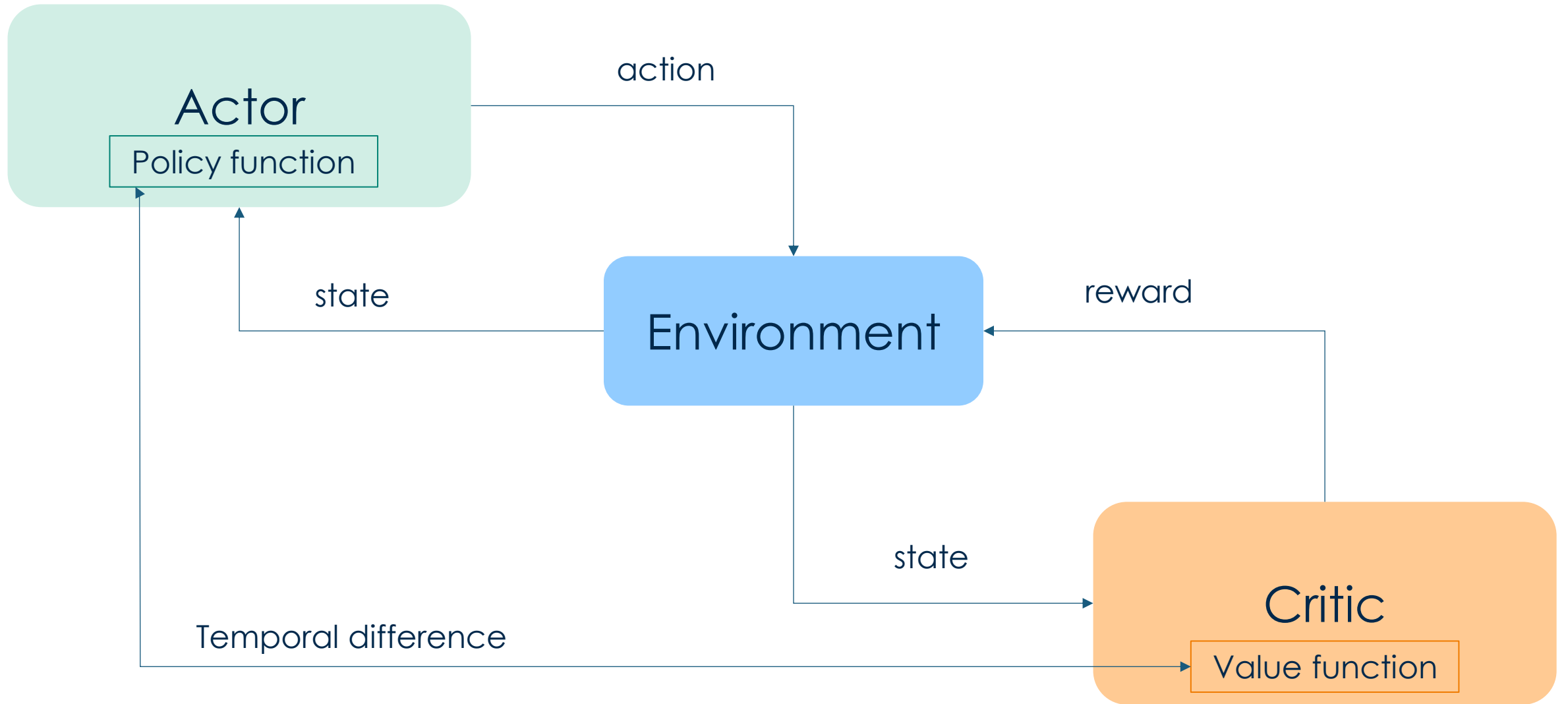
Policy gradient is the basis for many modern algorithms (REINFORCE, PPO)

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi} [Q^{\pi}(s, a) \nabla_{\theta} \log \pi(a|s; \theta)]$$

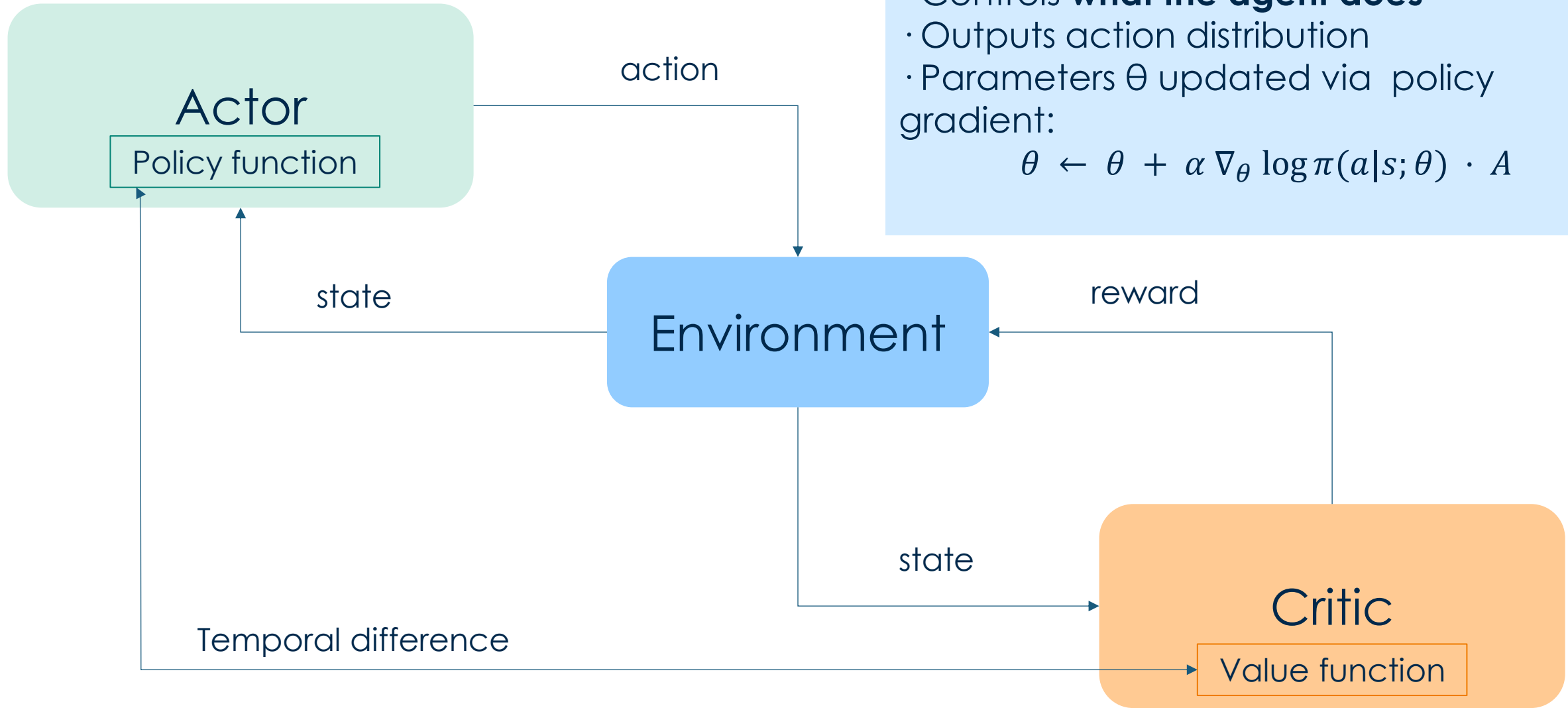
REINFORCE algorithm:

1. Sample episode $\tau = (s_0, a_0, r_1, \dots, s_t)$
2. For each t : $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$
3. Update: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi(a_t|s_t; \theta) \cdot G_t$
4. Repeat

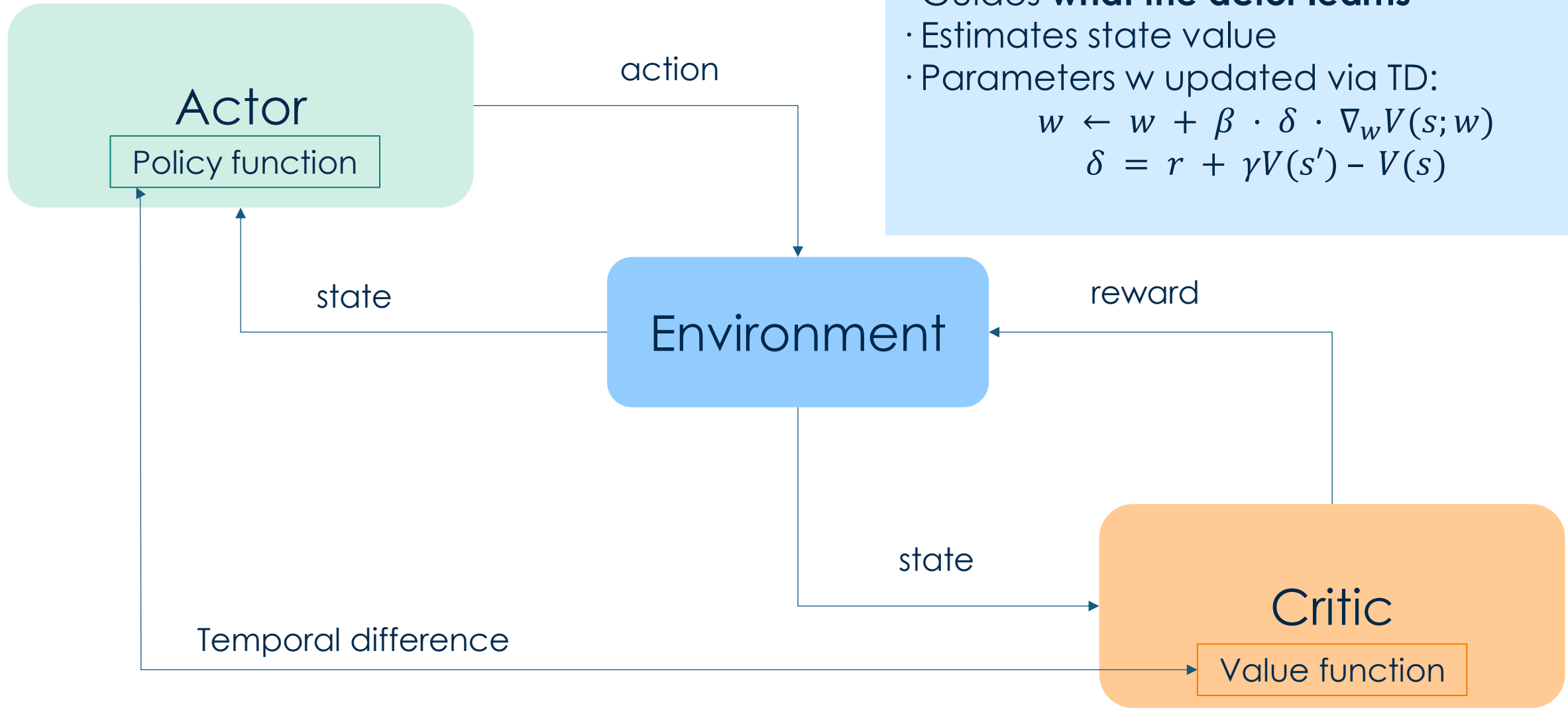
Actor-critic architecture



Actor-critic architecture



Actor-critic architecture



Soft-actor critic

Motivation; augment the reward with an entropy term — encourage the agent to be as random as possible while still collecting reward
 $J(\pi) = \sum_t \mathbb{E}[r(s_t, a_t) + \alpha H(\pi(\cdot | s_t))]$, with α temperature parameter

Off-Policy

Uses a replay buffer → highly sample efficient. Learns from old experience.

Continuous Control

Policy outputs mean + log_std of Gaussian.
Reparameterization trick for backprop.

Two Q-networks

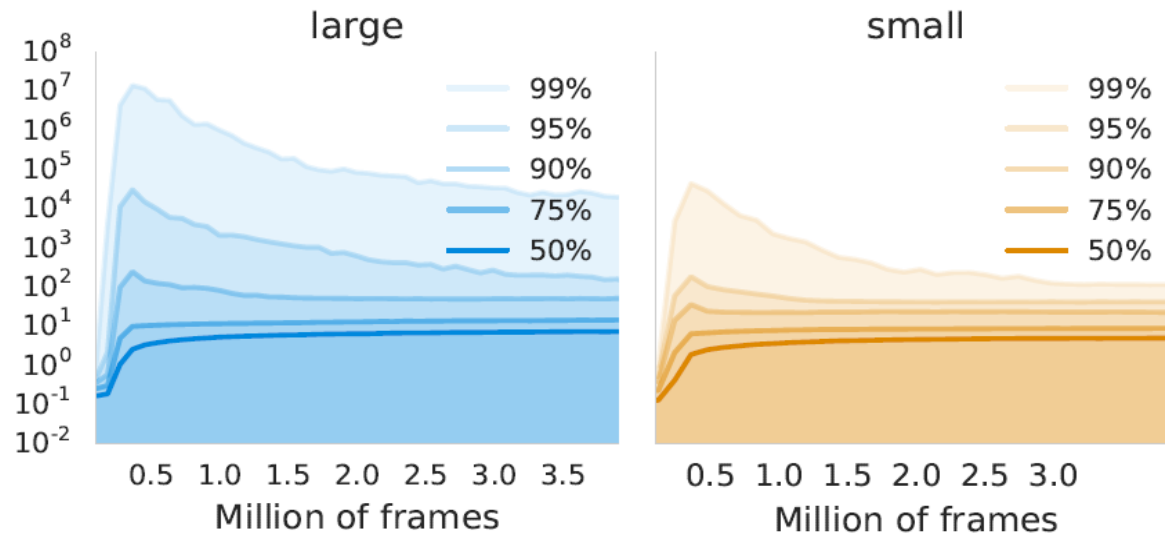
Twin critics: take min to reduce overestimation.

Auto Temperature

α adjusted online: if entropy > target → decrease α , else increase. No manual tuning.

Deadly triad in deep RL

Although divergence is a possibility, in practice soft divergence (Q value explosion followed by convergence) is more common

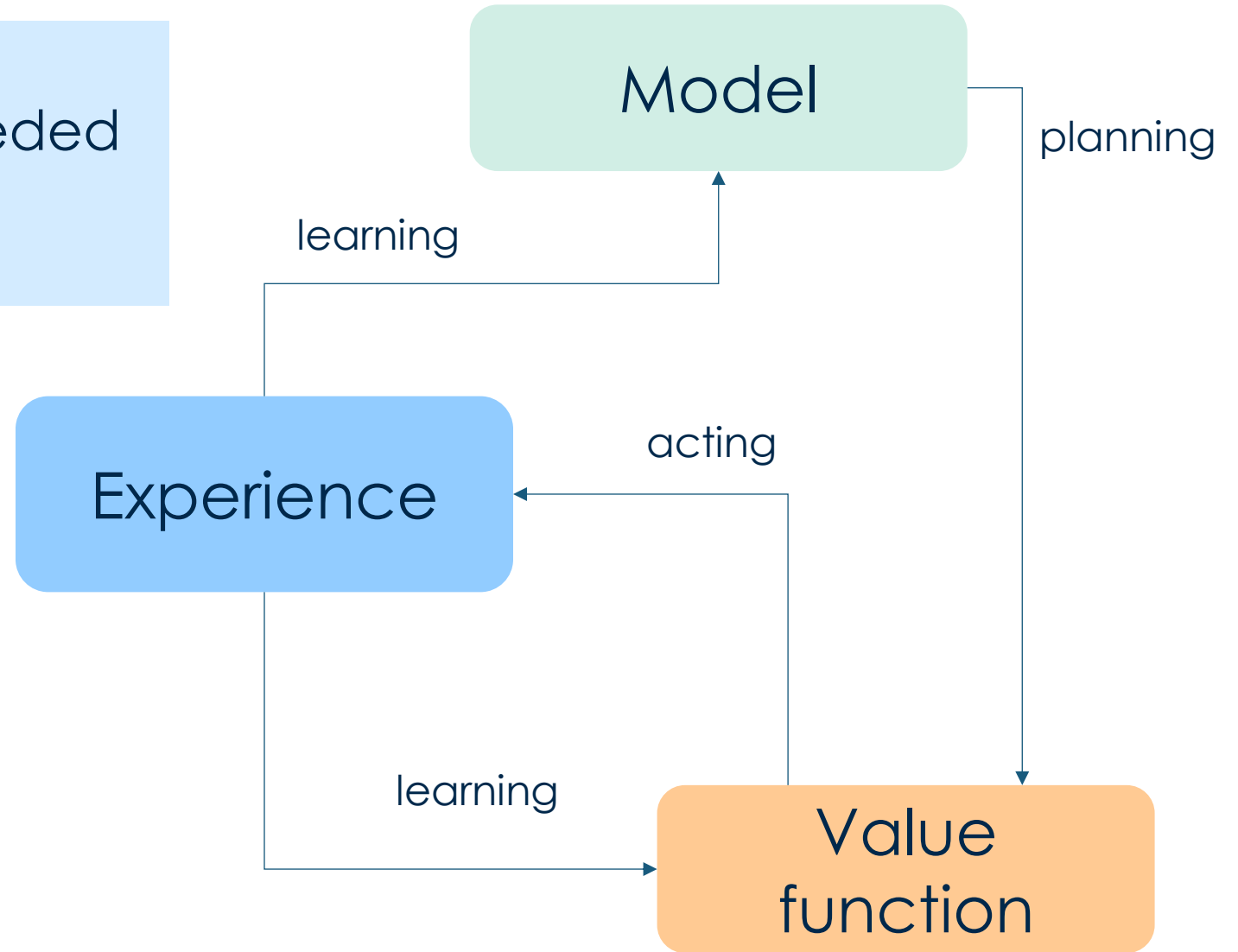


Van Hasselt, Hado, et al. "Deep reinforcement learning and the deadly triad." *arXiv preprint arXiv:1812.02648* (2018).

Model-based RL

Much more sample efficient
→ fewer real interactions needed
Model errors propagate
→ compounding mistakes

- Dyna: real experience + model rollouts
- Dreamer (2020): learn latent world model, train entirely in imagination
- AlphaZero: perfect model (game rules) + MCTS planning



Curiosity ~~killed~~ saved the cat

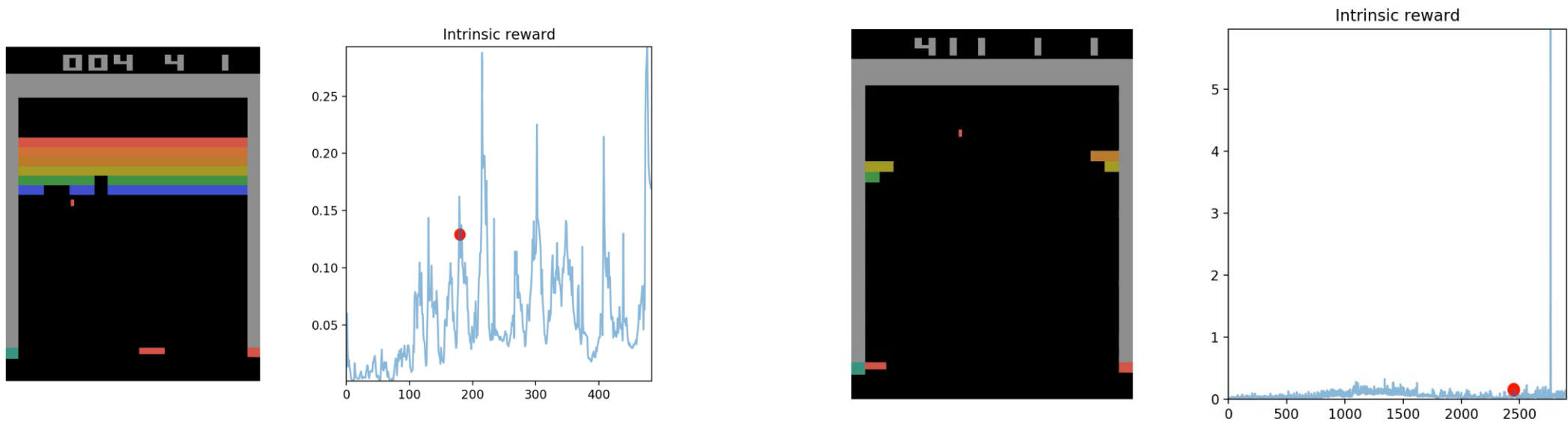
Modern RL suffers from two problems:

- the **sparse rewards or non-existing rewards problem**: in many problems, most intermediate rewards do not contain information and hence are set to zero.
- the **extrinsic reward function is hand-engineered**: it may be suboptimal or lead to unintended behaviours

The idea of curiosity-driven learning is to build a reward function that is **intrinsic** to the agent (that is generated by the agent itself, rather than by external environment). In this sense, the agent will act as a self-learner since it will be the student, but also provide its own feedback.

Curiosity as a form of exploration

Curiosity can be implemented as predicting the next state: the reward is the prediction error.



The agent experiences spikes of intrinsic reward when it sees a new configuration of bricks early on in training and when it passes the level for the first time after training for several hours.

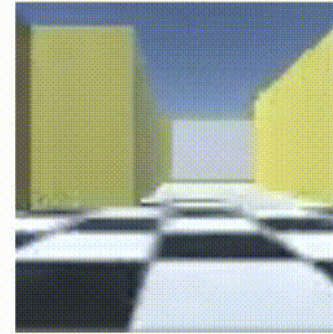
<https://openai.com/index/reinforcement-learning-with-prediction-based-rewards/>

Curiosity ~~saved~~ killed the cat

The “Noisy-TV” problem started as a thought experiment in [Burda, et al \(2018\)](#). If an RL agent is rewarded with seeking novel experience, a TV with unpredictable random noise ($\rightarrow$ high prediction error $\rightarrow$ high reward) would be able to attract the agent's attention forever.



Agent in a maze with a noisy TV



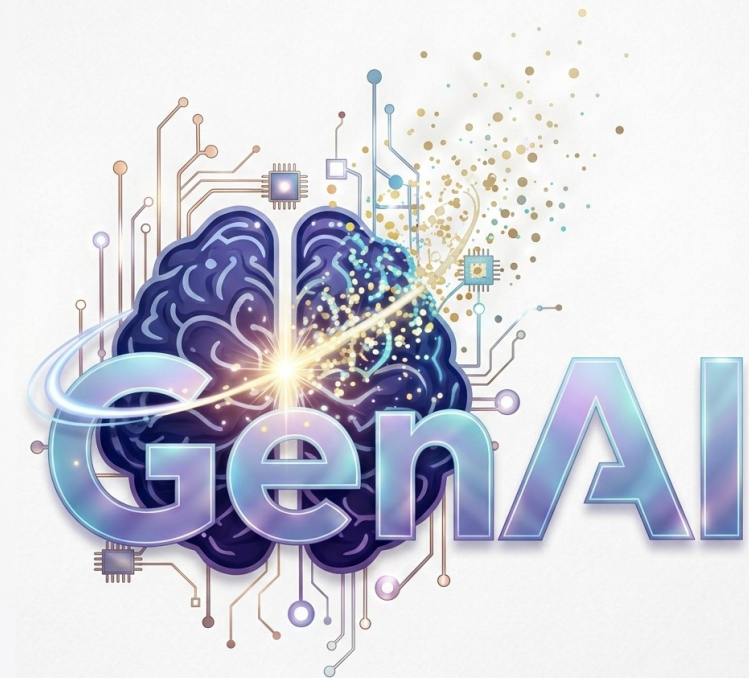
Agent in a maze without a noisy TV

(Image source: OpenAI Blog: "[Reinforcement Learning with Prediction-Based Rewards](#)")



Politecnico
di Torino

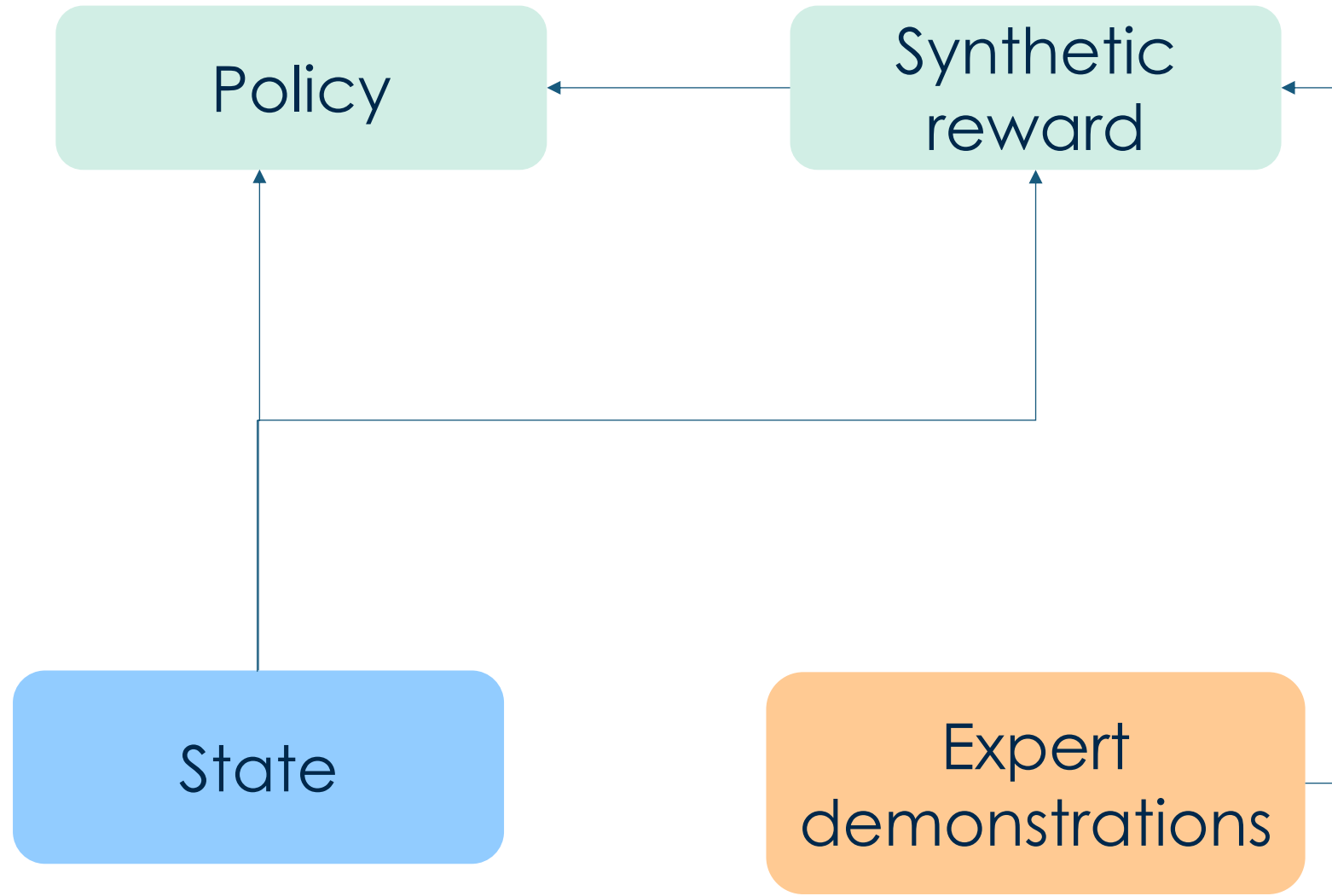
Learning from demonstration



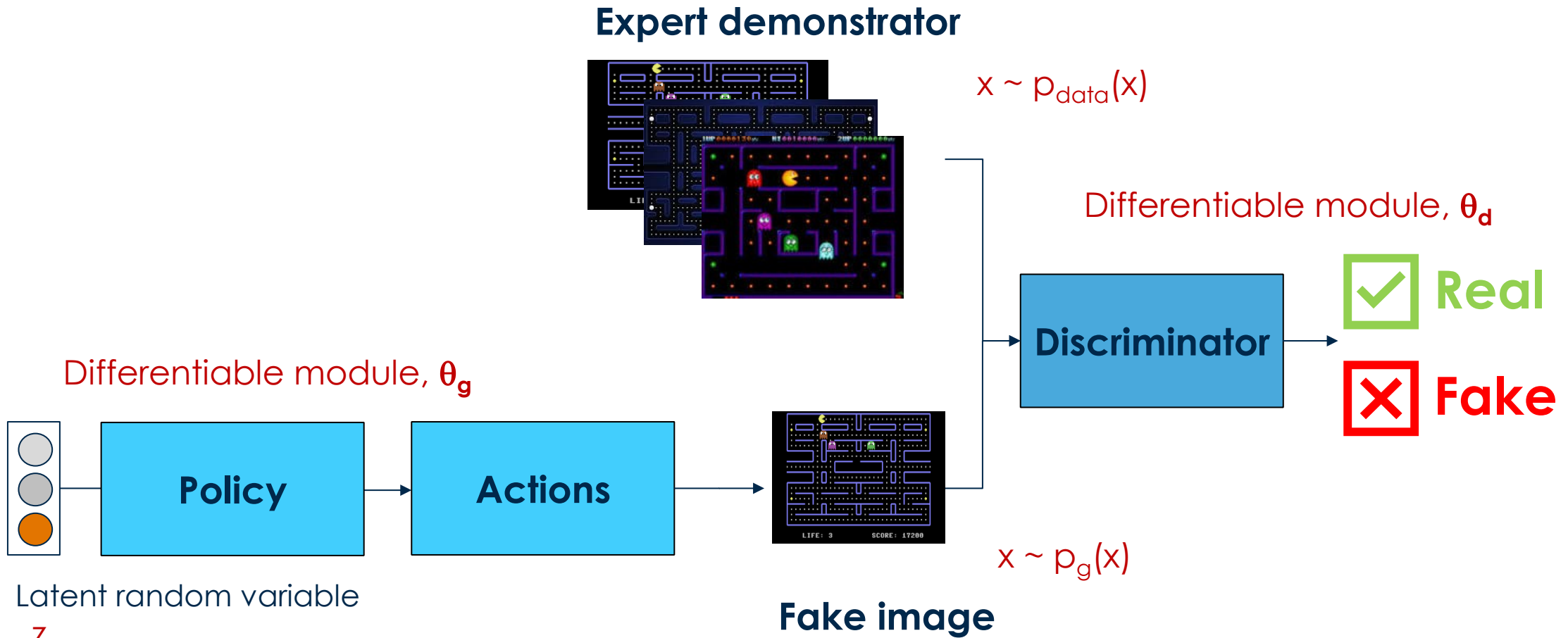
Imitation learning

- In imitation learning, the policy is learnt from expert demonstrations without access to reward function.
 - Behavioral cloning: supervised learning on (state, action) pairs. Simple but may overfit: $\pi_{\theta} = \operatorname{argmax}_{\theta} \sum \log \pi(a_{\text{exp}} | s_{\text{exp}}; \theta)$
 - Dagger: dataset aggregation — online correction by querying expert
 - Inverse RL (IRL): recover reward function from demonstrations

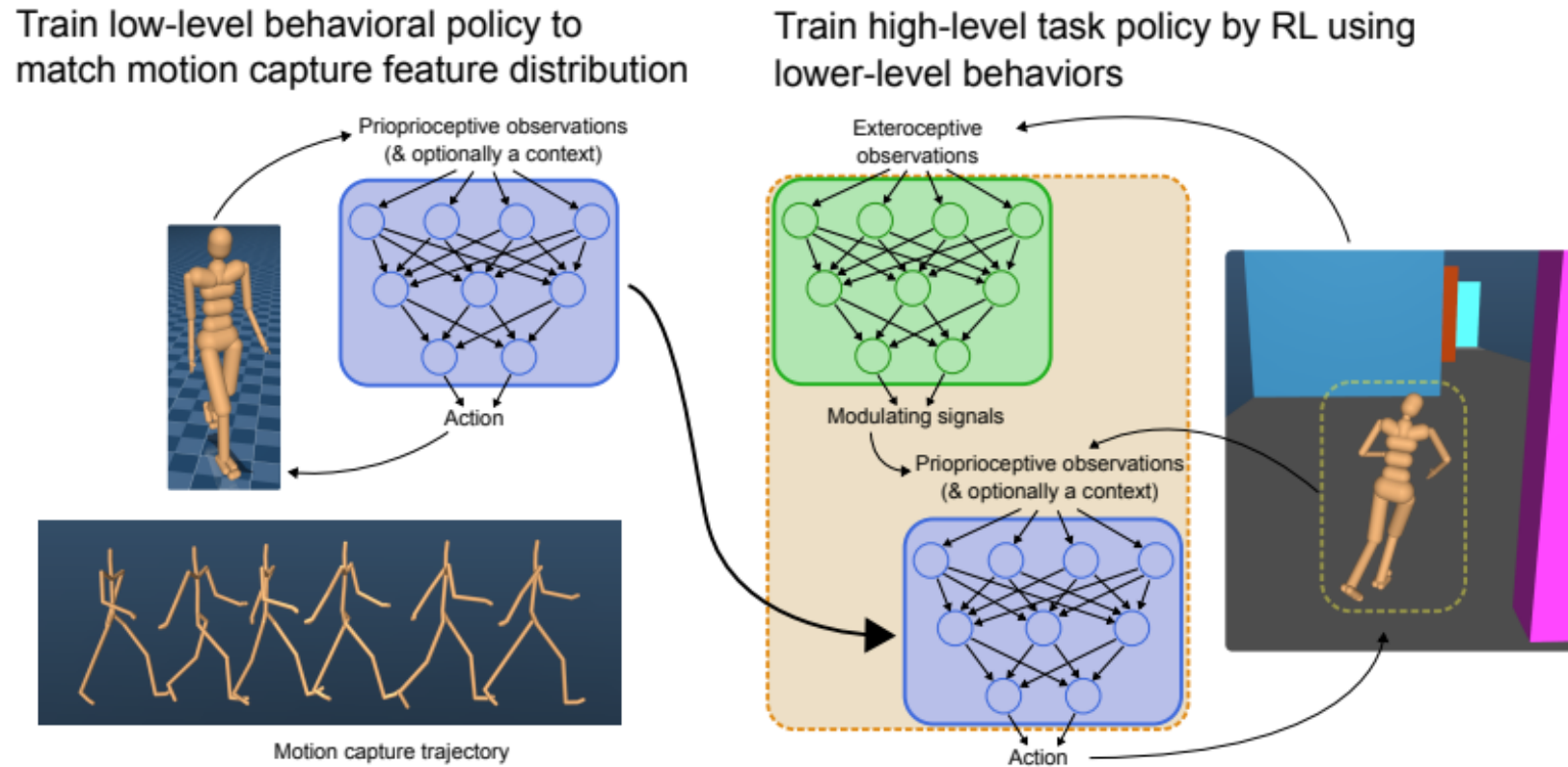
Generative Adversarial Imitation Learning



Generative Adversarial Imitation Learning

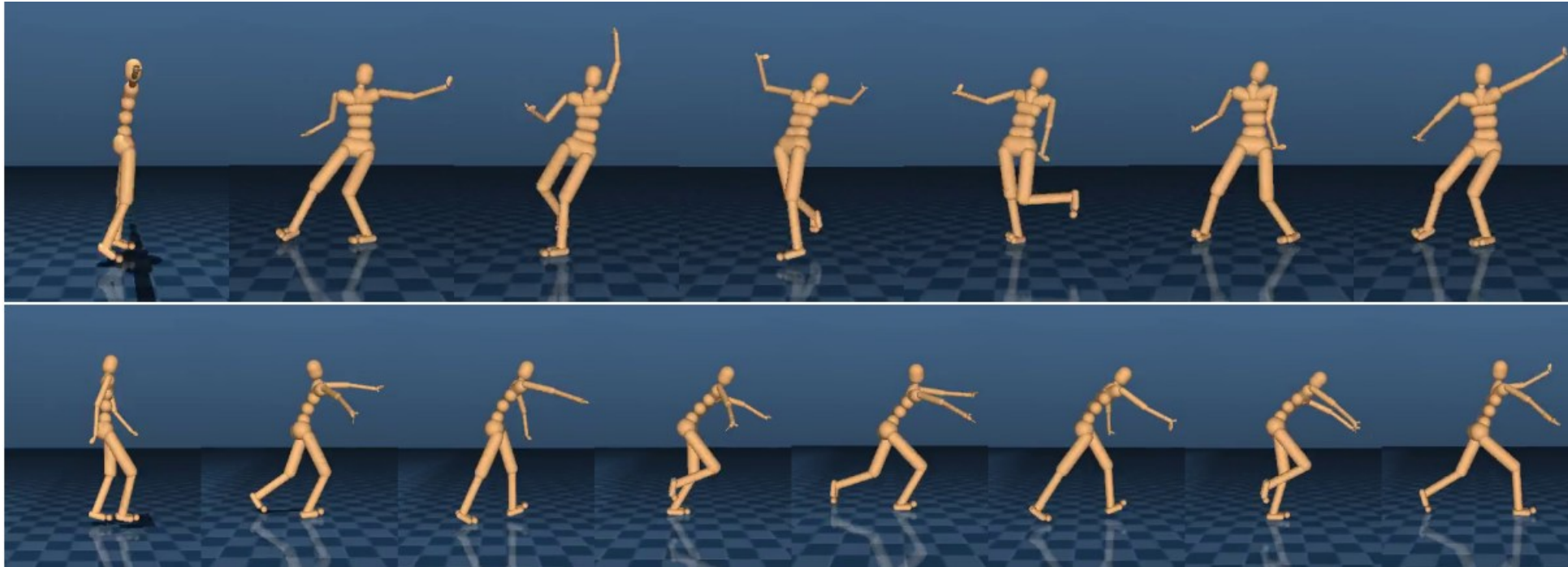


Example: learning human motion from MoCap



Merel, Josh, et al. "Learning human behaviors from motion capture by adversarial imitation." *arXiv preprint arXiv:1707.02201* (2017).

Random vs. MoCap initialization

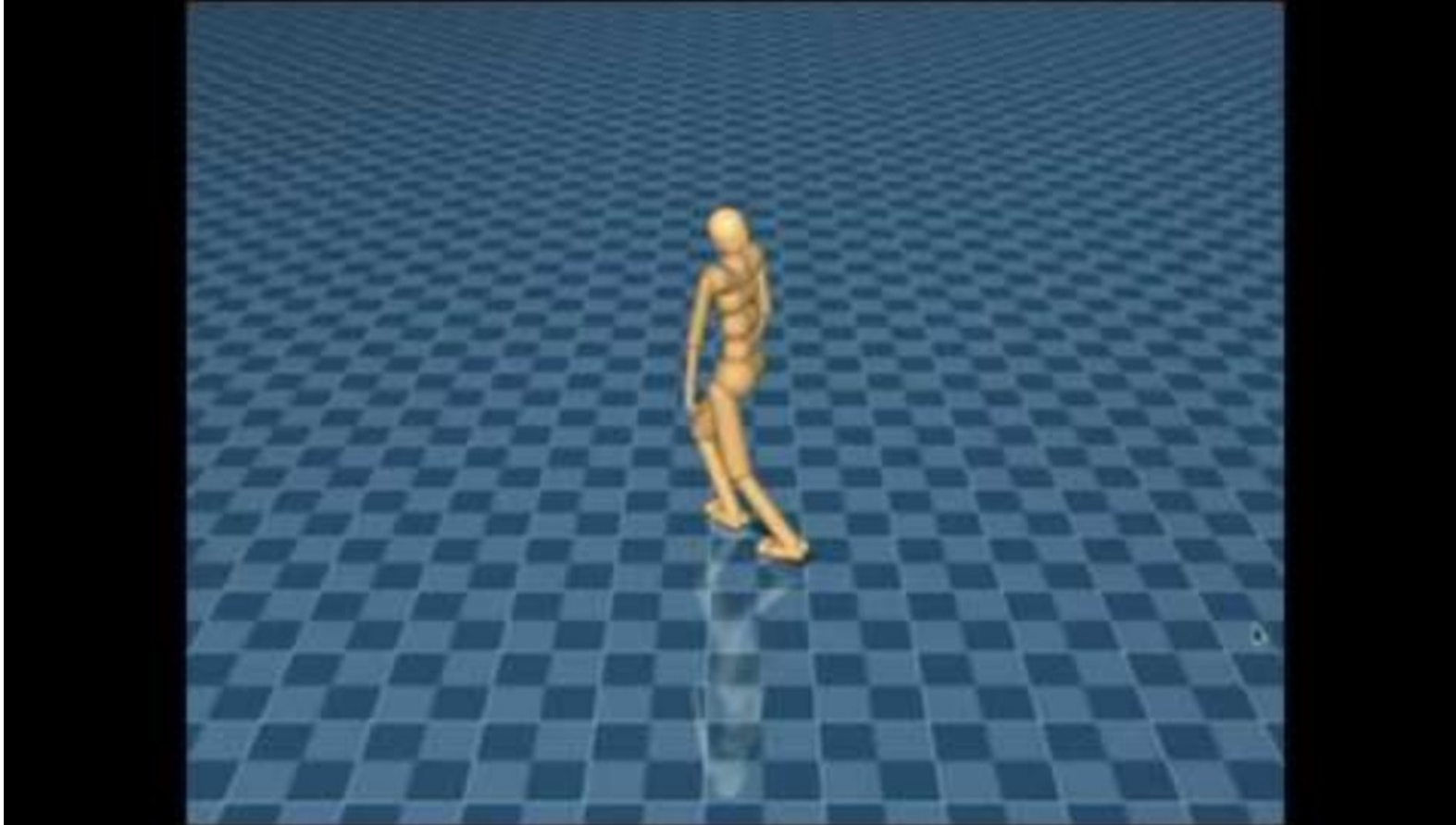


Random init →
unnatural
sideway skip

Behavioral
cloning →
forward motion

Merel, Josh, et al. "Learning human behaviors from motion capture by adversarial imitation." *arXiv preprint arXiv:1707.02201* (2017).

Example: learning human motion from MoCap



<https://www.youtube.com/watch?v=YsxN3uRBupc>